



(איזה כיף)



Elon Musk   @elonmusk · 32m

I ❤️ cyberpunk

...



Elon Musk   @elonmusk · 32m

I ❤️ cyberpunk

...





Competitive Programming At University of Haifa

Welcome to the Competitive Programming At University of Haifa website!

This website is designed to help students learn and practice competitive programming.

לפני שנתחיל - אנחנו צריכים שכולם יפתחו חשבון
באתר CODEFORCES

עכשיו ניתן כמה דקות למי שעדיין לא פתח חשבון,
לפתוח ולשלוח לאיתן את שם המשתמש שלהם
בפרטי

ומי שפתח צריך לבצע רק את הצעד השני

שקופית להיות בה לכמה דקות ✨



מה צפוי לנו במהלך השנה ?

לאט לאט נלמד איך לתכנת ב-CPP

```
helloworld.cpp X
helloworld.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <string>
4
5  using namespace std;
6
7  int main()
8  {
9      vector<string> msg{"Hello", "C++", "World", "from", "VS Code!", "and the C++ extension!"};
10     msg.
11     for assign
12     {
13         at
14     }
15     cout
16 }
```

void std::vector<std::__cxx11::string>::assign(std::size_t __n, const std::__cxx11::string &__val)

+2 overloads

@brief Assigns a given value to a %vector.
@param __n Number of elements to be assigned.
@param __val Value to be assigned.

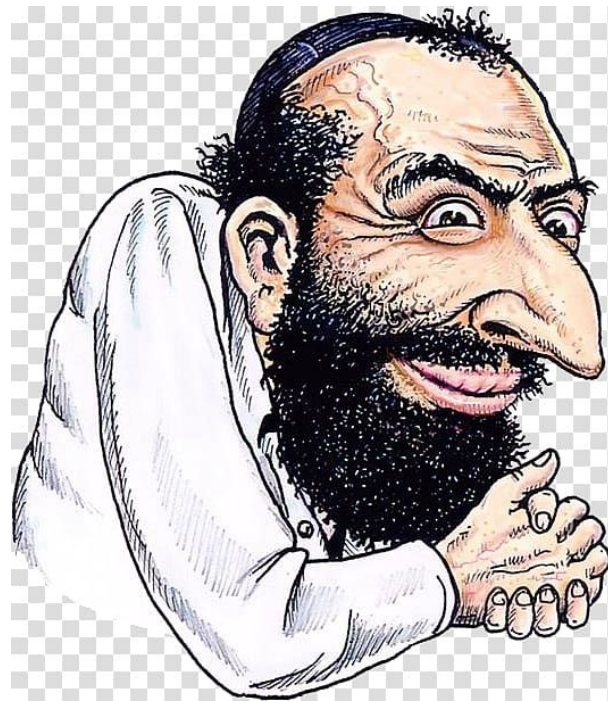
This function fills a %vector with @a __n copies of the given

נלמד שיטות ומבני נתונים שיעזרו לנו לפתור שאלות

(המרצה למבני נתונים)



(greedy algorithms)



בסוף כל מפגש נשלח לכם שאלות לתרגול באתר

(חלקן פתרנו בהרצאה ואתם תממשו)

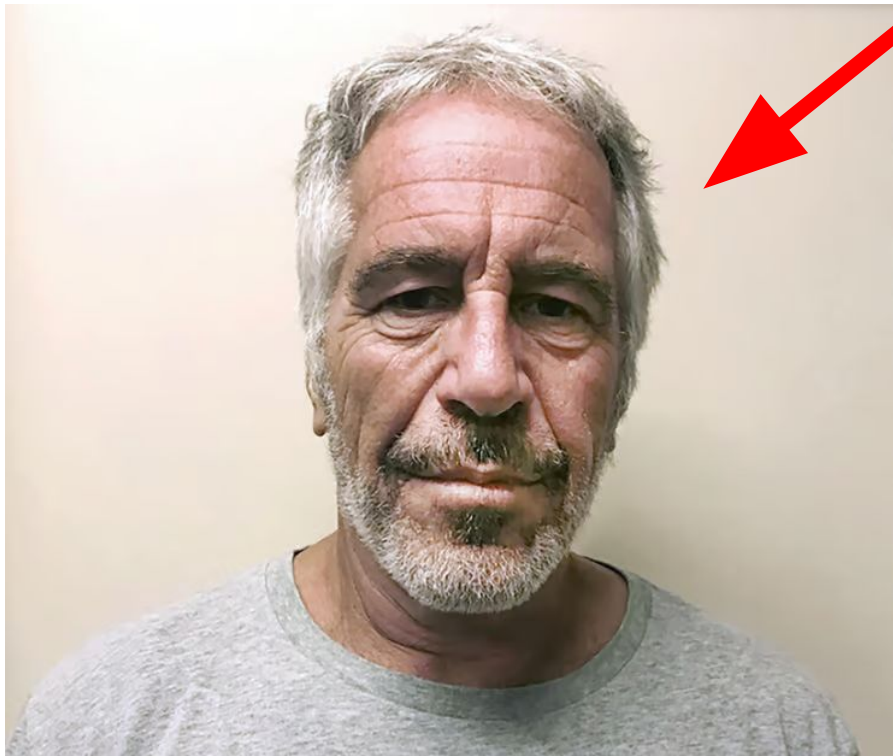
ומקום ראשון בסוף יקבל סוכרייה (אולי)

	Username	Total Score	1	2
0	amit	2	✓	✓
1	dansa	2	✓	✓
2	eldar	2	✓	✓
3	orva07	2	✓	✓
4	nadav	2	✓	✓
5	phantom	2	✓	✓
6	radagast	2	✓	✓
7	kishooa	0	-	-
8	dvir10205	0	-	-
9	eld	0	-	-
10	ohadguetta	0	-	-
11	avigail_tordjman	0	-	-
12	rotem	0	-	-
13	michaelperlin22	0	-	-
14	orihmelnits	0	-	-
15	ron_m14	0	-	-
16	seanb00	0	-	-
17	yanir	0	-	-
18	asafa121	0	-	-

השאלות לא חובה 🥲

אבל ככל שתתרגלו יותר יגדלו הסיכויים שתטוסו ותהיו מלכי ה CP 😁

אתם



ופעם בכמה מפגשים נעשה סימולציה של תחרות אמיתית



ומה המטרה בסוף ? ? ?

מטרה ראשונה - להכין אתכם לקורס בקיץ

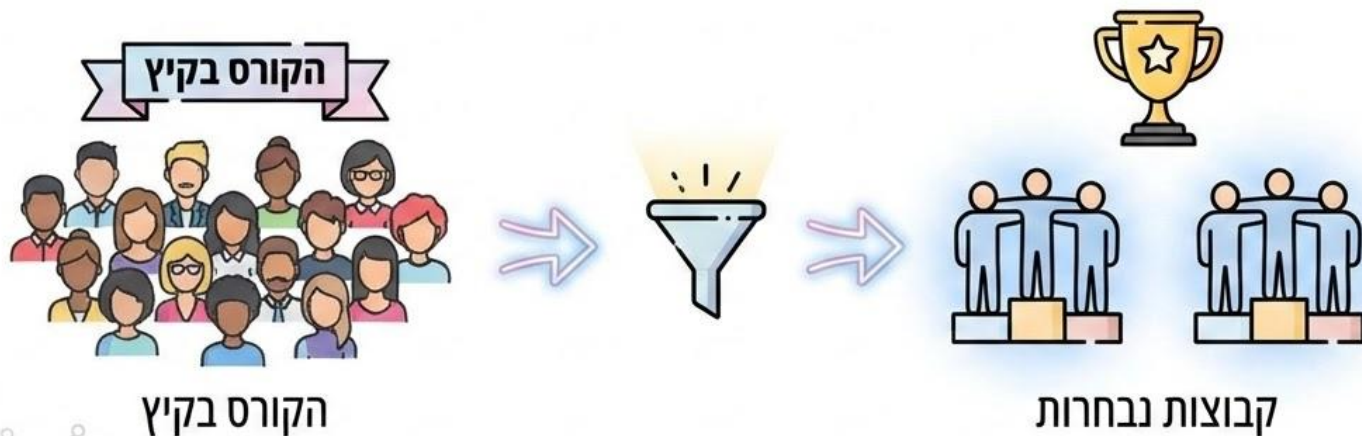
(שגם אותו אנחנו נעביר אז יש לכם פרוטקציות)

הציון שלכם

שם רכיב קורס	ציון
01ק203.8010 ציון סופי למועד	100



**אחרי הקורס בקיץ ישלחו 2 או 3 (אל תוסיפו לזה 4 🤔)
קבוצות בגודל של 3 אנשים, שהם יהיו האנשים הכי טובים בקורס.**



לכן חשוב להשקיע עכשיו שתקבלו
חופשה ממומנת בעתיד

מטרה שניה - להעלות את הרמה של הנבחרת שתישלח ל-swerc

שנהיה
לראש



1	 The Open University of Israel: Fire Code	11	1320	+	+2	+	+	+	+1	+1	-1	+	+	+2	+
				01:08	00:56	01:18	00:05	00:31	00:21	02:10		04:23	01:02	04:59	03:07

ולא לזנב



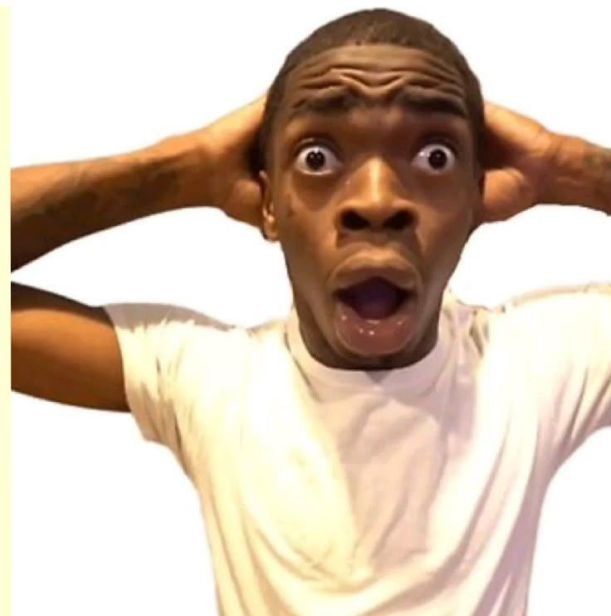
149	 Ecole Nationale Supérieure d'Électronique, Informatique, Télécommunications, Mathématique et Mécanique de Bordeaux: Enseirb	1	262				+6		-5						
							02:22								

והמטרה האמיתית היא

החברים שרכשנו לאורך הדרך



מה צריך לדעת כדי לפתור שאלת תכנות תחרותי ?



1. את הפתרון
2. לתכנת



Assembly



C



C++



Haskell



Java



Node.js



Pascal



Python3



Ruby



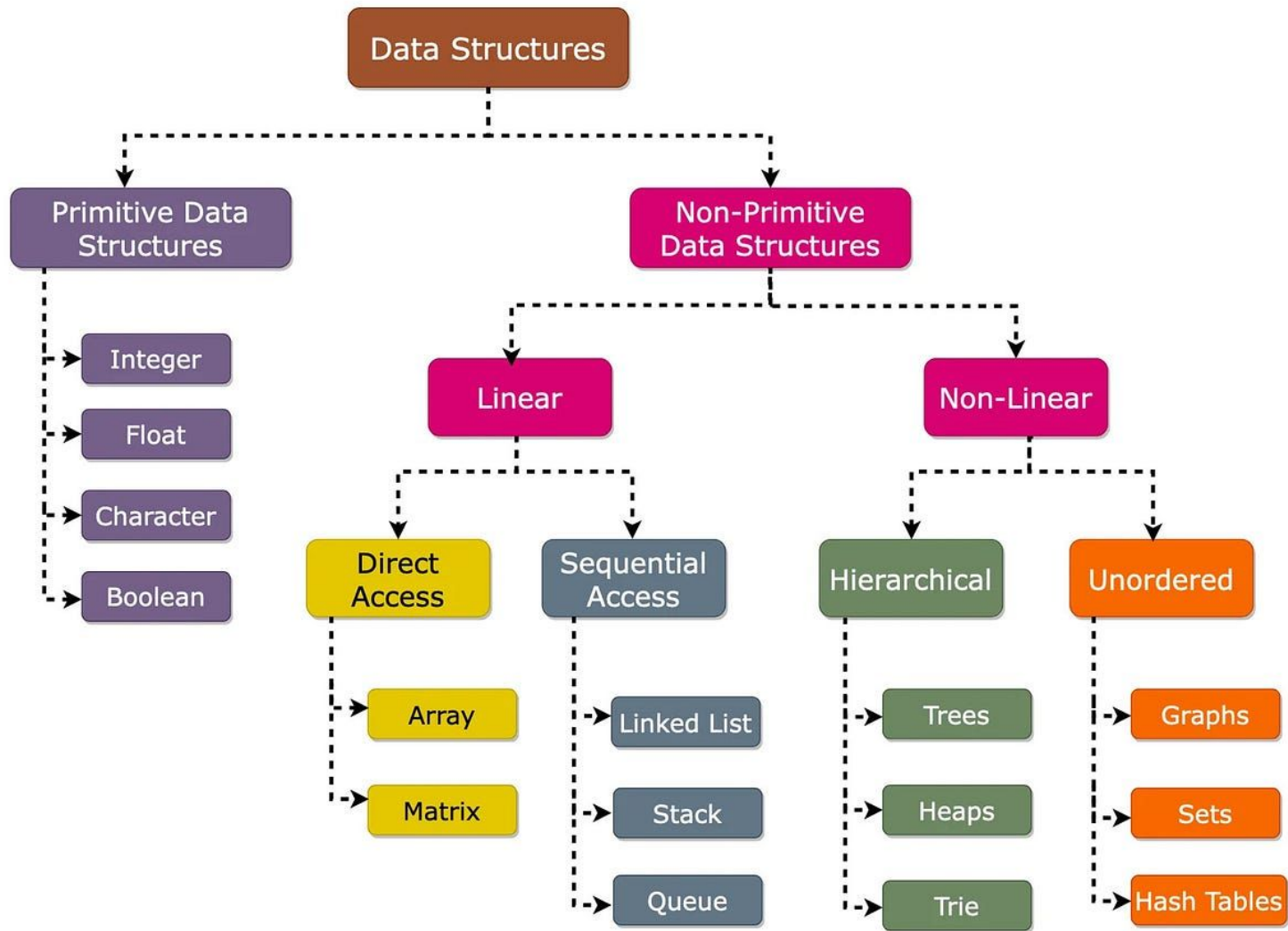
Rust



Scala



Scala

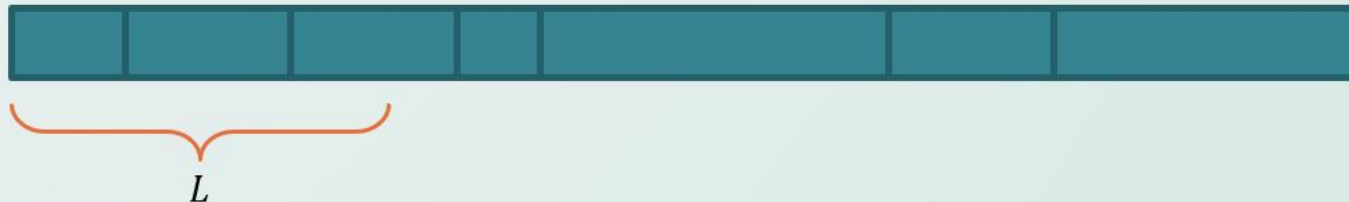


greedy

פשוט יותר ממה שזה נראה

בעיית הנגר

- נגר מקבל קורת עץ לחיתוך באורך כולל S .
- על הקורה מסומנים n מיקומים אפשריים לחיתוך $x_1, x_2, \dots, x_n$, שרק בהם יכול הנגר לחתוך את הקורה.
- מזמין העבודה זקוק לקורות באורך לכל היותר L . בפרט, הוא רוצה שלאחר החיתוך, כל חלקי הקורות יהיו לא יותר ארוכות מ- L .
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, שיבחר את מקומות החיתוך, כך שמספר הקורות באורך לכל היותר L יהיה מינימלי. הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.

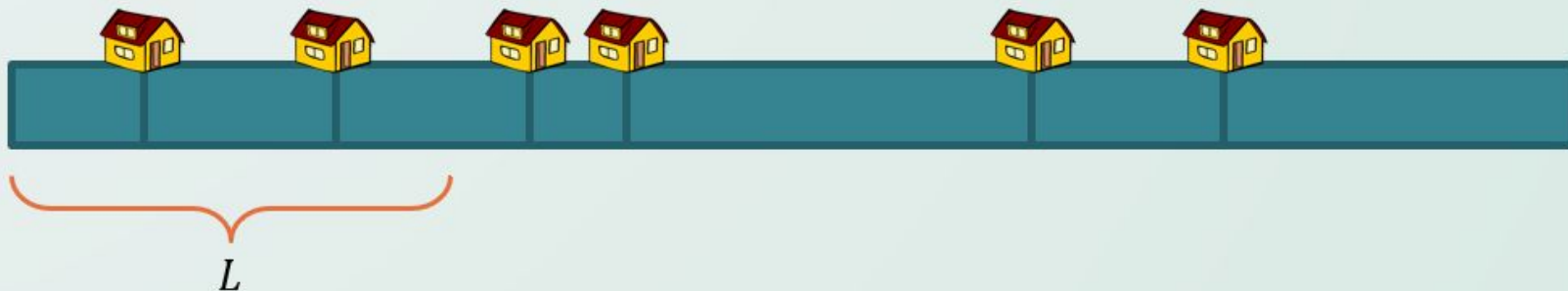


מהי בעיה חמדנית?

A greedy algorithm solves optimization problems by making the locally optimal choice at each step, hoping to find the global optimum. It is efficient but doesn't always guarantee the best overall solution. Key properties include the greedy choice property and optimal substructure.

סיפור אחר, אותה הבעיה

- ישנו מסלול באורך S ועליו מסומנות n נקודות עצירה שונות $x_1, x_2, \dots, x_n$.
- אתם יודעים כי בכל יום תוכלו ללכת מרחק של לכל היותר L . עליכם כעת לבחור את מקומות הלינה, מבין n נקודות העצירה השונות.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, שיבחר את מקומות העצירה, כך שתסיימו את המסלול במספר ימים מינימלי. הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.



E. Cakes

time limit per test: 1 second
memory limit per test: 256 megabytes



This summer, you plan to organize a large party and invite many friends. They have a sweet tooth, so you plan to bake nice cakes for them. You know the recipe for a nice chocolate cake, and you want to cook as many of them as possible.

Given the N ingredients needed to make a single cake and the ingredients that you have in your kitchen, how many cakes can you make?

Input

- The first line of the input contains a single integer N .
- Then, N lines follow, one for each ingredient. Each of these lines contains two positive integers: the first one is the required quantity of this ingredient per cake, the second one is the quantity of this ingredient you have in your kitchen.

textbf{Limits}

- $1 \leq N \leq 10$
- All ingredient quantities will be integers between 1 and 10 000.

Output

The output should contain a single integer: the maximum number of cakes you can make using the available ingredients.

Examples

input	Copy
3 100 500 2 5 70 1000	
output	Copy
2	

input	Copy
3 100 50 2 5 70 1000	
output	Copy
0	

תכירו, בעיית תכנות תחרותי

E. Cakes

time limit per test: 1 second

memory limit per test: 256 megabytes



This summer, you plan to organize a large party and invite many friends. They have a sweet tooth, so you plan to bake nice cakes for them. You know the recipe for a nice chocolate cake, and you want to cook as many of them as possible.

Given the N ingredients needed to make a single cake and the ingredients that you have in your kitchen, how many cakes can you make?

Input

- The first line of the input contains a single integer N .
- Then, N lines follow, one for each ingredient. Each of these lines contains two positive integers: the first one is the required quantity of this ingredient per cake, the second one is the quantity of this ingredient you have in your kitchen.

Limits

- $1 \leq N \leq 10$
- All ingredient quantities will be integers between 1 and 10 000.

Output

The output should contain a single integer: the maximum number of cakes you can make using the available ingredients.

Examples

input

Copy

```
3
100 500
2 5
70 1000
```

output

Copy

```
2
```

input

Copy

```
3
100 50
2 5
70 1000
```

output

Copy

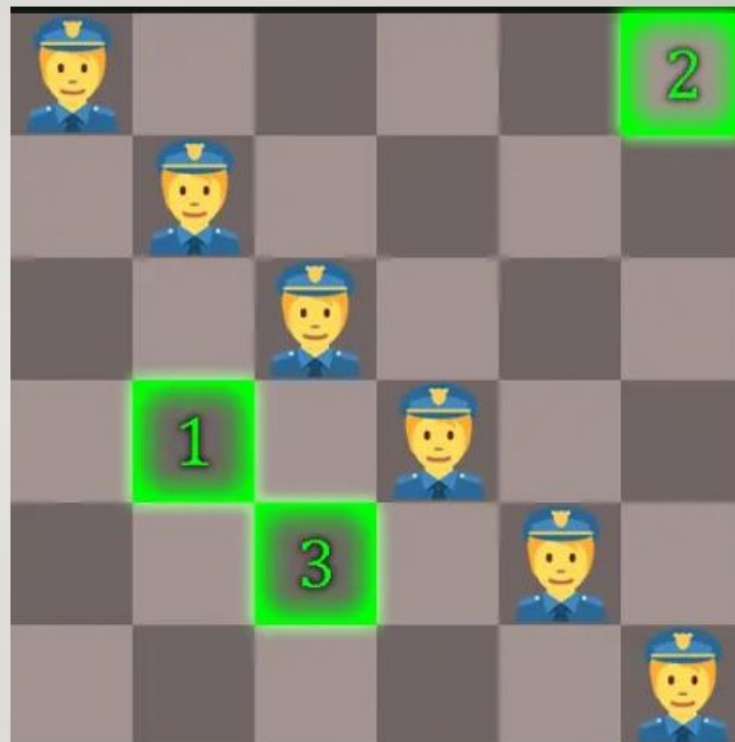
```
0
```

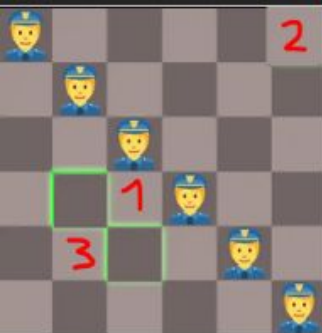

BASRENG

- אתה רוצה למכור את הסחורה הגנובה שלך בBasreng, עיר שיכולה להיות מיוצגת כמטריצה בגודל $2n$ על $2n$.
- באלכסון הראשי של המטריצה יש שוטרים, מותר לך שכל שוטר יראה אותך פעם אחת, אם הוא יראה אותך פעמיים אז תיתפס.
- המטרה שלך היא לבחור נקודות כך שכל שוטר רואה אותך פעם אחת (נקודה אחת בעמודה או בשורה שלו), ומרחק ההליכה שלך אם תלך מנקודה 1 ל-2 ל-3 ל-4... הוא מקסימלי אפשרי. אם יש כמה פתרונות אפשריים תדפיס אחד. אי אפשר ללכת באלכסון.



- זאת דוגמה עבור $n=3$. אתה צריך לבחור 3 נקודות כך שמרחק ההליכה שלך יהיה כמה שיותר גדול, אם אתה הולך לנקודות לפי הסדר, אין מעברים באלכסון.
- בדוגמה הזאת מרחק ההליכה הוא 14, זאת אחת הדרכים האופטימליות עבור $n=3$.
- מצאו בזמן $O(n)$ דרך אופטימלית והדפיסו אותה.

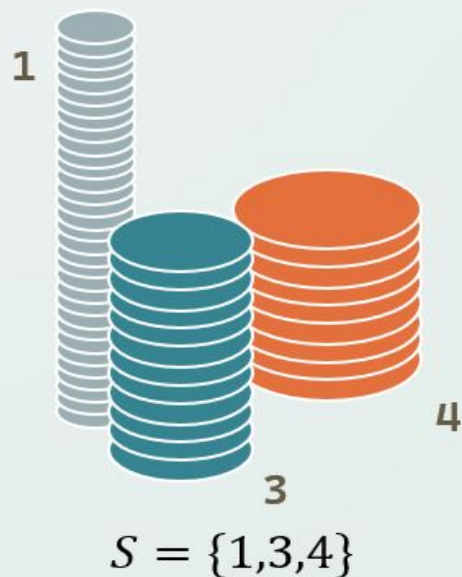




פתרון

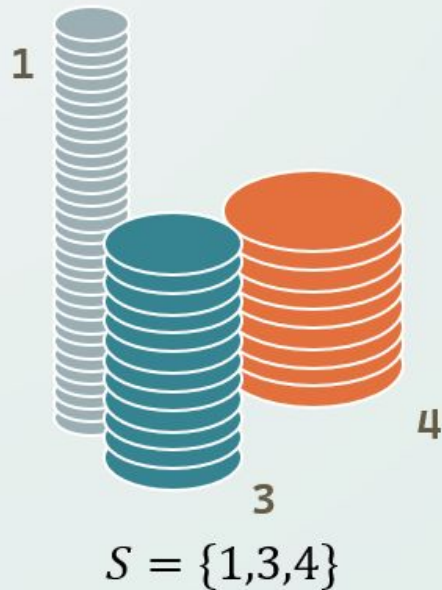
- ברור שאופטימלי לבחור נקודה בצד ימין למעלה של האלכסון הראשי, והבאה בצד ימין למטה שלו... כלומר לא לבחור פעמיים ברצף נקודה באותו צד של השוטרים.
- הבחנה 1- קיים פתרון אופטימלי בו כל הנקודות שבחרת הם על האלכסון המשני. לא קשה לראות את זה אז לא נתעכב על ההוכחה, ונניח שכל הנקודות באלכסון המשני כי זה יעזור.
- הבחנה 2- ניתן להציג את התוספת של כל נקודה בהליכה בתור המרחק לבוא אליה מהחצי שלה של הלוח (של השוטרים), ועוד המרחק לחזור ממנה אל החצי השני של הלוח, מלבד הנקודה הראשונה והאחרונה כי אליהן לא הולכים או לא חוזרים.
- פתרון- נבחר את הנקודה הראשונה להיות נקודה קרובה לשוטרים באלכסון המשני, אז נבחר את הנקודה הכי רחוקה בצד השני של האלכסון המשני, אז נבחר את הנקודה הכי רחוקה שניתן באלכסון המשני... ונעשה ככה ל-n נקודות. זה כמו את הנקודה הראשונה לשים קרוב, ומאז לבחור כל פעם את הנקודה הכי רחוקה שאפשר בצד ההפוך של האלכסון המשני

הגדרה לבעיית המטבעות



- נתונה קבוצה של k ערכי מטבעות שלמים שונים $S = \{c_1, c_2, \dots, c_k\}$
- בנוסף נתון סכום מטרה n .
- נרצה להרכיב את הסכום n בעזרת מטבעות מתוך הקבוצה S .
- מהו מספר המטבעות המינימלי שנצטרך?
- באילו מטבעות נשתמש כדי לייצג את הסכום בצורה מינימלית?
- האם בכלל אפשר לייצג את הסכום n בעזרת המטבעות?

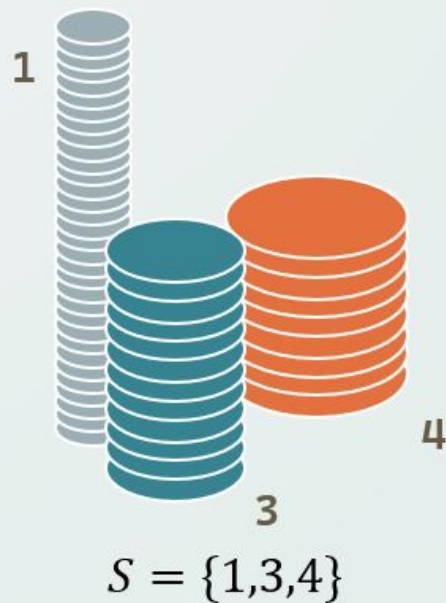
הפתרון החמדני



- פתרון אפשרי הינו הפתרון החמדני הבא:
- ניקח כל פעם את המטבע הכי גדול שהוספתו אינה חורגת מהסכום
- האם הפתרון יעבוד? כיצד נוכל לדעת אם לא ניתן להרכיב את הסכום n ?

הפתרון החמדני

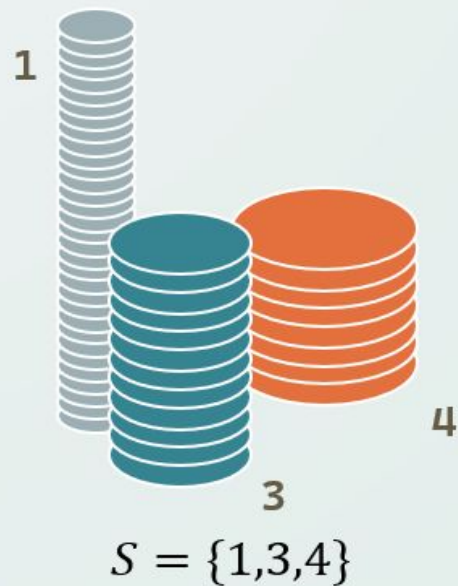
- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו- $n = 6$:



נותר לסכום: 6

הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו- $n = 6$:



נותר לסכום: 2



הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו- $n = 6$:

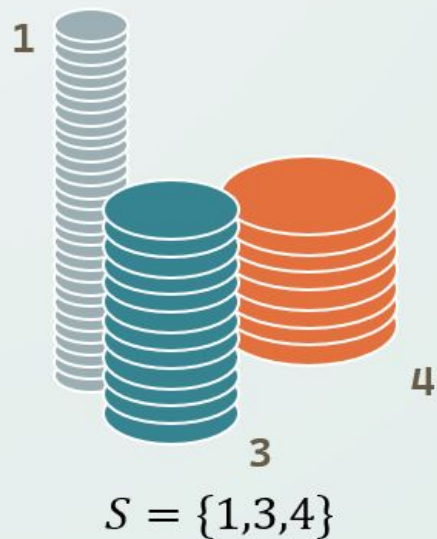


נוטר לסכום: 1

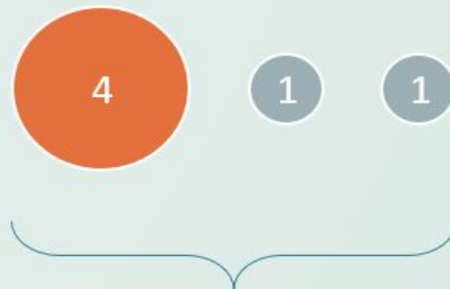


הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו- $n = 6$:



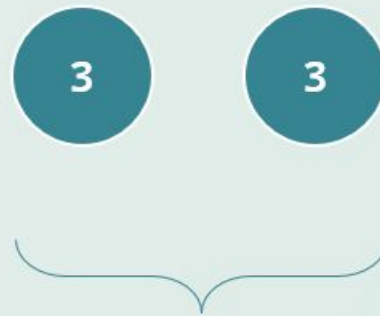
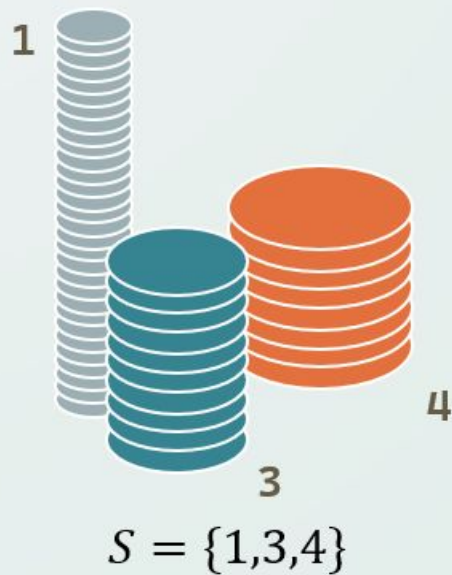
נותר לסכום: 0



הפתרון החמדני נתן סכום
בעזרת 3 מטבעות

הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו- $n = 6$:



ישנו פתרון טוב יותר בעזרת 2
מטבעות בלבד!

למישהו יש רעיון אחר איך לפתור את זה בצורה חמדנית?



DP(dynamic programming)

ובשפת הקודש: תכנון דינמי

מה זה בכלל תכנון דינמי?

הרעיון המרכזי מאחורי תכנות דינמי הוא מאוד פשוט: נשמור את הפתרונות עבור תת-בעיות קטנות, ונשתמש בהם כדי לפתור בעיות גדולות יותר – ובכך נחסוך חישובים מיותרים ונשפר משמעותית את זמן הריצה



עובדה לא כל כך מעניינת: התכנון הדינאמי הומצא על ידי יהודי ולשם dynamic programming אין כמעט משמעות זה בעיקר נשמע מגניב.

דוגמא

סדרת פיבונאצ'י

למי שלא מכיר, סדרת פיבונאצ'י מוגדרת באופן הבא:

$$F_n = F_{n-1} + F_{n-2}$$

$$F(0) = 0$$

$$F(1) = 1$$

כאשר:

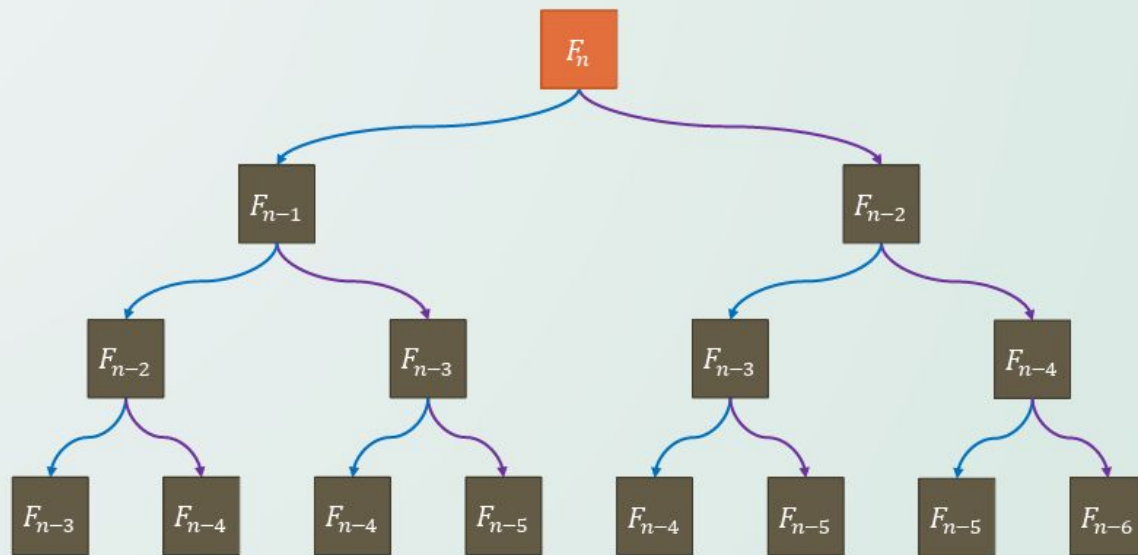
נפתור באופן נאיבי:

נרצה לחשב את האיבר ה-n בסדרה, ונכתוב קוד הפותר רקורסיבית באופן נאיבי את הבעיה:

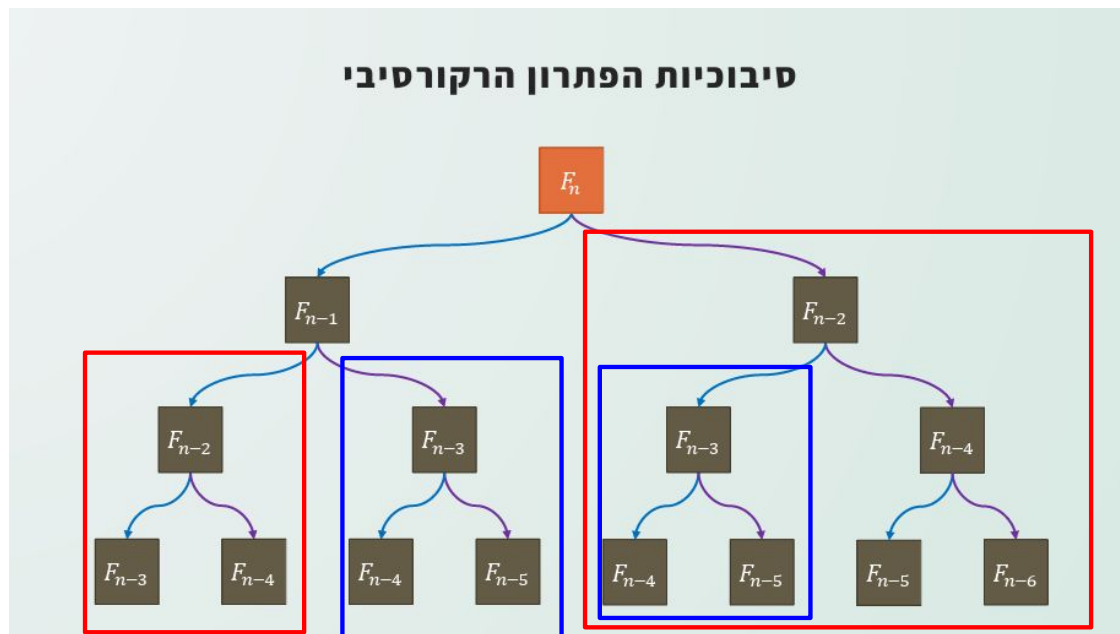
```
int fib(int n) {  
    if (n<2) return 1;  
    return fib(n-1)+fib(n-2);  
}
```


עץ הקריאות הרקורסיבי יראה כך:

סיבוכיות הפתרון הרקורסיבי



נשים לב כי נבצע חישובים של אותו האיבר מספר פעמים ללא סיבה



סיבוכיות נוראית: $O(n^2)$

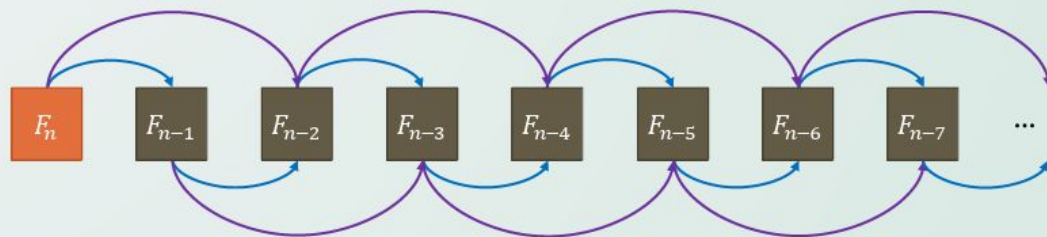


נשנה את הקוד כך שנשמור במערך חישובים שכבר בוצעו

```
vector<int> fib_mem(n:1e5, value:-1);  
int fib(int n) {  
    if (fib_mem[n] != -1) return fib_mem[n];  
    if (n<2) return n;  
    return fib_mem[n] = fib_mem[n-2]+fib_mem[n-1];  
}
```

הקריאות של הקוד החדש יראו כך:

מרקורסיה לאיטרציה



סיבוכיות מעולה: $O(n)$



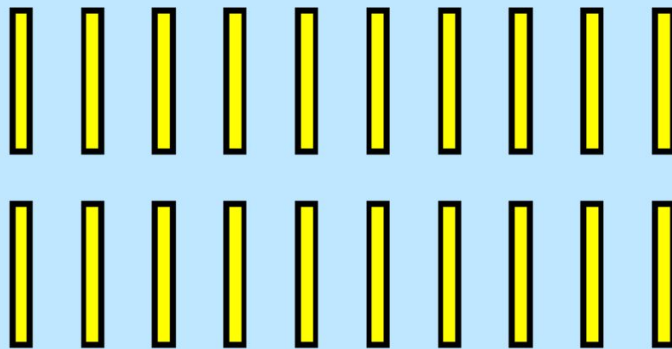
למי שמתעניין - ניתן לצמצם את השימוש בזיכרון ל- $O(1)$

```
int fib(int n) {  
    int a, b, c;  
    a = 0;  
    b = 1;  
    for(int i = 2; i <= n; i++) {  
        c = a + b;  
        a = b;  
        b = c;  
    }  
    return c;  
}
```

דוגמא נוספת - 20 גפרורים

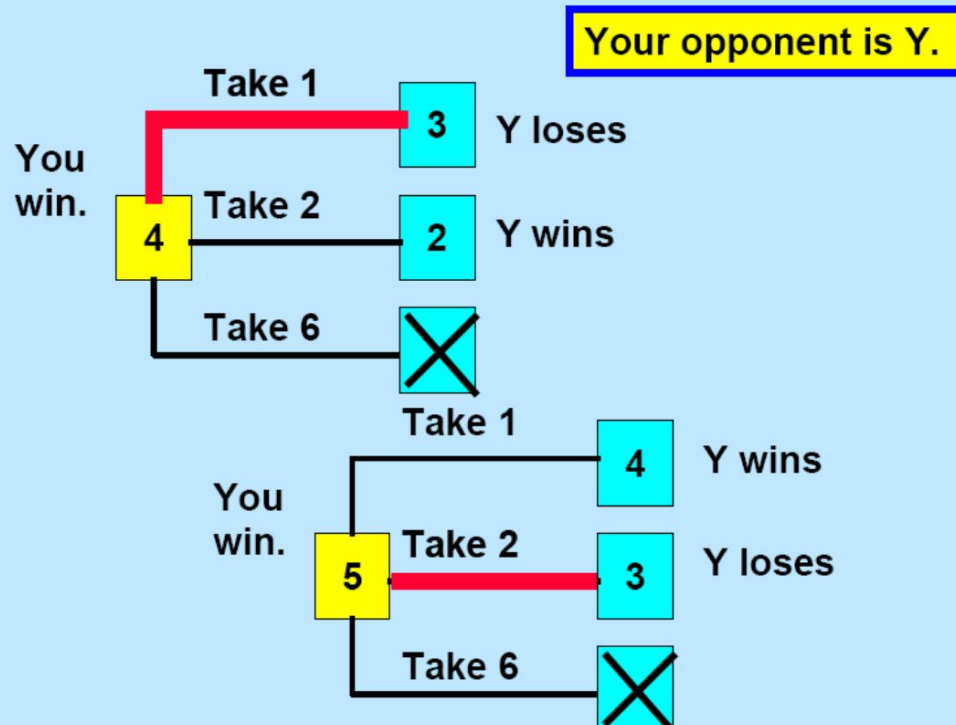
במשחק 20 גפרורים נרצה להחזיר מי מהשחקנים מנצח בסיבוכיות אופטימלית, נגדיר את המשחק באופן הבא:

- Choose a person to go first.
- Alternate in eliminating 1 or 2 or 6 matches.
- The person who takes the last match wins.



עץ החלטות עם מספר גפרורים של 4 או 5

Decisions with 4 or 5 matches left.



עץ החלטות עם מספר גפרורים של 6 או 7

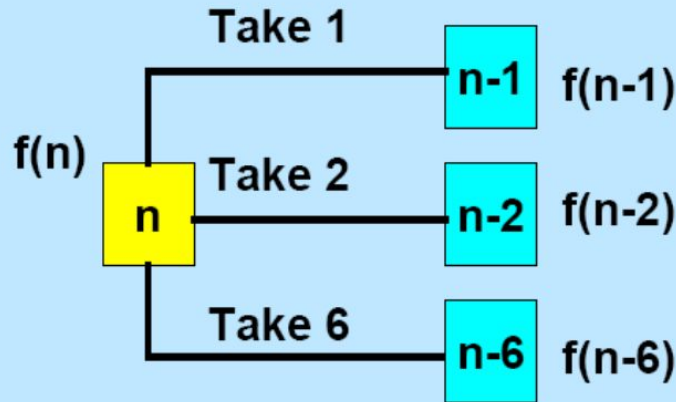


Decisions with 6 or 7 matches left.



נפתור בעזרת תכנון דינמי:

Dynamic Programming Recursion



The Dynamic Programming
Recursion

$f(n) = W$ if $f(n-1) = L$
or $f(n-2) = L$
or $f(n-6) = L$;

$f(n) = L$ if $f(n-1) = f(n-2) = f(n-6) = W$.

איך הטבלה תראה עבור 50 גפרורים:

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

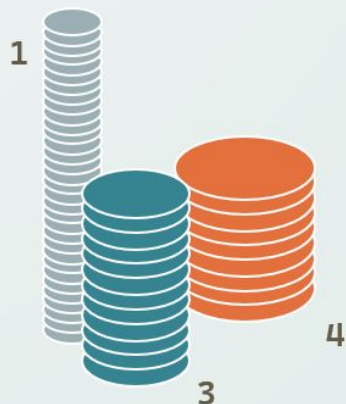
סיבוכיות מעולה: O (מספר הגפרורים)



נחזור לשאלת המטבעות:

נפתור בסיבוכיות: $O(kn)$

הגדרה לבעיית המטבעות



$$S = \{1, 3, 4\}$$

- נתונה קבוצה של k ערכי מטבעות שלמים שונים $S = \{c_1, c_2, \dots, c_k\}$
- בנוסף נתון סכום מטרה n .
- נרצה להרכיב את הסכום n בעזרת מטבעות מתוך הקבוצה S .
- מהו מספר המטבעות המינימלי שנצטרך?
- באילו מטבעות נשתמש כדי לייצג את הסכום בצורה מינימלית?
- האם בכלל אפשר לייצג את הסכום n בעזרת המטבעות?

נפתור בעזרת DP:

נשים לב שהפתרון האופטימלי עבור x בטוח יהיה הפתרון האופטימלי של x-coin מסוים, ועוד 1 כי נוסיף את המטבע הזה. לכן נעבור בלולאה על המספרים מ1 עד n, וכל אחד נעדכן להיות האופטימלי מבין הקודמים אליו בהפרש מטבע אחד.

```
using vi = vector<int>;
const int oo = 1e18;

signed main(){
    cin.tie(0)->sync_with_stdio(sync:false), cout.tie(0);
    int n, x, c; cin >> n >> x;
    vi dp(n+1, oo); dp[0]=0;
    rep(i, 0, n){
        cin >> c;
        rep(j, 0, x-c+1) if(dp[j] != oo) dp[j+c] = min(dp[j+c], dp[j]+1);
    } cout << (dp[x] == oo ? -1 : dp[x]);
}
```

עכשיו תורכם לפתור שאלה

C. Long Jumps

time limit per test: 2 seconds

memory limit per test: 256 megabytes

Polycarp found under the Christmas tree an array a of n elements and instructions for playing with it:

- At first, choose index i ($1 \leq i \leq n$) — starting position in the array. Put the chip at the index i (on the value a_i).
- While $i \leq n$, add a_i to your score and move the chip a_i positions to the right (i.e. replace i with $i + a_i$).
- If $i > n$, then Polycarp ends the game.

For example, if $n = 5$ and $a = [7, 3, 1, 2, 3]$, then the following game options are possible:

- Polycarp chooses $i = 1$. Game process: $i = 1 \xrightarrow{+7} 8$. The score of the game is: $a_1 = 7$.
- Polycarp chooses $i = 2$. Game process: $i = 2 \xrightarrow{+3} 5 \xrightarrow{+3} 8$. The score of the game is: $a_2 + a_5 = 6$.
- Polycarp chooses $i = 3$. Game process: $i = 3 \xrightarrow{+1} 4 \xrightarrow{+2} 6$. The score of the game is: $a_3 + a_4 = 3$.
- Polycarp chooses $i = 4$. Game process: $i = 4 \xrightarrow{+2} 6$. The score of the game is: $a_4 = 2$.
- Polycarp chooses $i = 5$. Game process: $i = 5 \xrightarrow{+3} 8$. The score of the game is: $a_5 = 3$.

Help Polycarp to find out the maximum score he can get if he chooses the starting index in an optimal way.

Input

The first line contains one integer t ($1 \leq t \leq 10^4$) — the number of test cases. Then t test cases follow.

The first line of each test case contains one integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of the array a .

The next line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — elements of the array a .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output on a separate line one number — the maximum score that Polycarp can get by playing the game on the corresponding array according to the instruction from the statement. Note that Polycarp chooses any starting position from 1 to n in such a way as to maximize his result.

נבנה את ה DP:

נגדיר:

$dp[i]$ = score starting at position i

כלומר הערך באיבר ה- i יהיה הערך של a במקום ה- i ועוד הערך של dp באיבר שקופצים אליו:

$$dp[i] = a[i] + dp[i+a[i]]$$

נמלא את המערך מהסוף להתחלה על פי הנוסחא.

פתרון

```
void solve()
{
    int n;
    cin>>n;
    vector<int> a(n);
    rep(i,0,n) cin>>a[i];

    vector<int> score(n);
    for(int i=n-1;i>=0;i--) {
        score[i]=a[i];
        int j = i+a[i];
        if(j<n)
            score[i]+=score[j];
    }
    int maxval = 0;
    rep(i,0,n) {
        if(score[i]>maxval)
            maxval=score[i];
    }
    cout<<maxval<<endl;
}
```

B. Lecture Sleep

time limit per test: 1 second

memory limit per test: 256 megabytes

Your friend Mishka and you attend a calculus lecture. Lecture lasts n minutes. Lecturer tells a_i theorems during the i -th minute.

Mishka is really interested in calculus, though it is so hard to stay awake for all the time of lecture. You are given an array t of Mishka's behavior. If Mishka is asleep during the i -th minute of the lecture then t_i will be equal to 0, otherwise it will be equal to 1. When Mishka is awake he writes down all the theorems he is being told — a_i during the i -th minute. Otherwise he writes nothing.

You know some secret technique to keep Mishka awake for k minutes straight. However you can use it **only once**. You can start using it at the beginning of any minute between 1 and $n - k + 1$. If you use it on some minute i then Mishka will be awake during minutes j such that $j \in [i, i + k - 1]$ and will write down all the theorems lecturer tells.

Your task is to calculate the maximum number of theorems Mishka will be able to write down if you use your technique **only once** to wake him up.

Input

The first line of the input contains two integer numbers n and k ($1 \leq k \leq n \leq 10^5$) — the duration of the lecture in minutes and the number of minutes you can keep Mishka awake.

The second line of the input contains n integer numbers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^4$) — the number of theorems lecturer tells during the i -th minute.

The third line of the input contains n integer numbers $t_1, t_2, \dots, t_n$ ($0 \leq t_i \leq 1$) — type of Mishka's behavior at the i -th minute of the lecture.

Output

Print only one integer — the maximum number of theorems Mishka will be able to write down if you use your technique **only once** to wake him up.

פתרון

```
void solve()
{
    int n,k;
    cin>>n>>k;
    vector<int> a(n);
    vector<int> t(n);
    vector<int> pre(n, value:0);

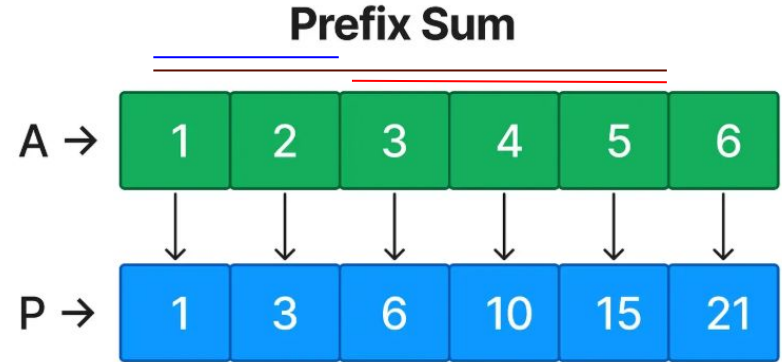
    rep(i,0,n) cin>>a[i];
    rep(i,0,n) cin>>t[i];

    int sum=0;

    rep(i,0,n)
        sum+=(t[i])?a[i]:0;

    pre[0] = (!t[0])?a[0]:0;
    rep(i,1,n)
        pre[i] = (!t[i])?a[i]+pre[i-1]:pre[i-1];

    int max_p = 0;
    rep(i,0,n-k+1) {
        int r = i+k-1;
        max_p = max(a[max_p], pre[r]-(i ? pre[i-1]:0));
    }
    cout<<max_p+sum<<endl;
}
```



אז זהו להיום